Tema 2

Optimización

Técnicas de optimización 1-dimensional

Índice

Esquema 2

Ideas clave 3

2.1 Introducción 3

2.2 Objetivos 4

2.3 Búsqueda dicotómica 5

2.4 Búsqueda de la Sección Áurea 8

2.5 Búsqueda de Fibonacci 13

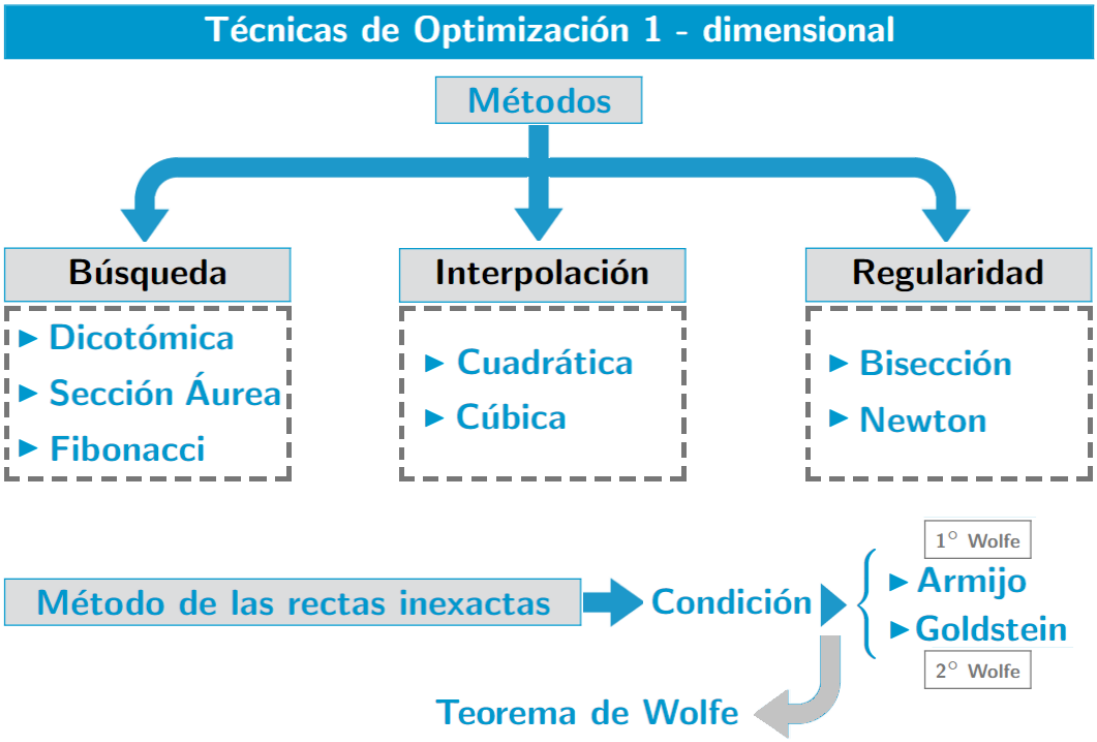
2.6 El método de la Interpolación Cuadrática 18

2.7 El método de Interpolación Cúbica 23

2.8 Método de la bisección 24

2.9 El método de las Rectas Inexactas 29

Esquema


Técnicas de Optimización 1 - dimensional
Métodos
Búsqueda
Dicotómica
Sección Áurea
Fibonacci
Interpolación
Cuadrática
Cúbica
Regularidad
Bisección
Newton
Método de las rectas inexactas
Condición
1° Wolfe
Armijo
Goldstein
2° Wolfe
Teorema de Wolfe

Ideas clave

2.1 Introducción

En el tema anterior, se han repasado algunos tipos de problemas de optimización. En particular, hay una división muy clara entre problemas unidimensionales, problemas multidimensionales sin restricciones y problemas multidimensionales con restricciones. La dificultad de estos problemas es creciente según el orden en que han sido enumerados. Una estrategia común en optimización (y en las matemáticas, en general) es resolver los problemas más complicados reduciéndolos a versiones más sencillas. En particular, los problemas multidimensionales con restricciones se encaran reformulándolos en problemas multidimensionales sin restricciones. De similar manera, es posible resolver problemas multidimensionales a problemas unidimensionales.

Así pues, una gran parte de los algoritmos para resolver problemas de optimización se basan en minimizar una función de coste de una variable. Esto descubre la optimización unidimensional como la piedra angular de la disciplina y pone de manifiesto la necesidad de saber resolver estos problemas de forma eficiente.

Un problema de **optimización unidimensional** consiste en minimizar una función de una variable. Esto es:

$$\text{minimizar } f(x) \ , \ x \in [a, b]$$

donde f es una función unimodal (tiene un único extremo relativo[1]) de variable real. Se denotará por x^* el valor donde se añlcanza el mínimo de la función f. Hay dos tipos de enfoques estandarizados para encontrar a x^*: Los métodos de **búsqueda** y los métodos de **aproximación**.

[1] Sin pérdida de generalidad, se asumirá que este extremo relativo es un mínimo.

Algunas características importantes para estos métodos, se pueden describir de la siguiente manera. En los métodos de búsqueda:

- Se genera una sucesión de intervalos $I_k = [a_k, b_k]$ a los cuales pertenece el elemento x^*, que realiza el mínimo .
- Si la longitud del intervalo I_k $[m(I_k)]$ es menor que una tolerancia preestablecida, entonces es posible asumir que cualquier elemento $x \in I_k$ es una aproximación suficientemente buena del valor donde se alcanza el mínimo.
- Típicamente se elige el punto medio del intervalo original, para el iterado inicial.
- Para aplicar este tipo de métodos, no es necesario requerir la derivabilidad en la función f.

Por rotra parte, para los métodos de aproximación, se puede destacar que se utiliza una aproximación de la función f, que resulta ser típicamente un polinomio. La idea, es aproximar la función f con alguna función mucho más sencilla para la cual, calcular su mínimo, es una operación elemental. Cabe destacar, que este tipo de enfoques requieren de cierto grado de derivabilidad (regularidad) de la función f.

2.2 Objetivos

Quizá, como objetivos principales se pueden destacar:

- Reconocer e implementar diversos algoritmos de búsqueda unidimensional-
- Determinar patrones para escogencia de ciertos parámetros relevantes en estos procesos, tales como: iterado inicial, criterio de parada (tolerancia), etc.
- Reconocer y construir iteraciones en base a estructuras establecidas a partir del conocimiento previo: Sucesión de Fibonacci, teorema de Bolzano, teorema de Weierstrass, Principio de intervalos encajados, etc.

Aunque, no se descartan ir añadiendo objetivos secundarios de tal manera que se pueda profundizar aún más en las implementaciones de estas técnicas en problemas más complicados y que sirvan de eje para resolver algunos problemas de la optimización multivariante.

2.3 Búsqueda dicotómica

Para el método de la **búsqueda dicotómica**, se considera una función continua y unimodal con un mínimo en el intervalo $I = [a, b]$. Este intervalo I será el *rango de incertidumbre*. Como ya se ha dicho, el valor donde se alcanza el mínimo x^* se localizará reduciendo el tamaño del rango de incertidumbre hasta que sea suficientemente pequeño. Tal procedimiento debe implementarse de manera que se verifique en cada nuevo intervalo que $x^* \in I_k$, para todo k. En este sentido, resulta necesario poder evaluar la función f en los intervalos.

Se eligen entonces $x_a, x_b \in I$ valores cualesquiera, tales que $x_a < x_b$ y se procede a evaluar f en esos puntos. Entonces, puede ocurrir alguno de los siguientes tres casos: distintas:

- **Caso 1** $f(x_a) < f(x_b)$
  Como el mínimo es el único extremo relativo (recuerde que la función es unimodal), necesariamente debe ocurrir que: $x^* \in [a, x_b]$. En efecto, si $f(x_a) < f(x_b)$ entonces x_b no puede realizar el mínimo y si la función fuese decreciente para $x > x_b$, se entra en contradicción con que la función sea unimodal.
- **Caso 2** $f(x_a) > f(x_b)$
  Caso totalmente análogo al **Caso 1**, por tanto debe suceder que: $x^* \in [x_a, b]$.
- **Caso 3** $f(x_a) = f(x_b)$
  En este caso y por lo visto en los dos anteriores, necesariamente debe suceder que: $x^* \in [x_a, x_b]$.

Este razonamiento proporciona los datos necesarios para realizar un planteamiento del algoritmo de la búsqueda dicotómica, el cual simplificaremos en cinco (5) sencillos pasos. Veamos a que se hace referencia:

- **Paso 1** **Valores de Entrada**

  Se considera el rango de incertidumbre $I = [a, b]$ y se fija una "pequeña" cantidad $\varepsilon > 0$ y se determina la tolerancia $\delta > 0$ para medir la precisión de la aproximación.

- **Paso 2** **Extremos del primer intervalo**

  Los primeros elementos a evaluar en la función f, vienen determinados por:

$$\boxed{x_a = x_1 - \frac{\varepsilon}{2}} \quad \text{y} \quad \boxed{x_b = x_1 + \frac{\varepsilon}{2}}$$

  donde x_1 es el punto medio del intervalo $[a, b]$, es decir $x_1 = \frac{a+b}{2}$. La cantidad $\varepsilon > 0$ debe ser lo sufientemente pequeña para así garantizar que $a \leq x_a$ y $x_b \leq b$.

- **Paso 3** **Evaluaciones**

  Se evalúan en la función los valores obtenidos en el **Paso 2** y se realizan las comparaciones de estas alturas.

- **Paso 4** **Escogencia del primer iterado**

  Si en el paso anterior sucede que $f(x_a) < f(x_b)$ se hace $\boxed{b = x_b}$, es decir el nuevo intervalo es de la forma: $I_1 = [a, x_b]$. Si por el contrario sucede que $f(x_a) > f(x_b)$, entonces se toma $a = x_a$ y se considera como nuevo intervalo a $I_1 = [x_a, b]$.

- **Paso 5** **Criterio de Parada**

  Se comprueba que la longitud del intervalo obtenido verifica el criterio de parada $\delta > 0$, es decir que $m(I_1) < \delta$. Si no se cumple se vuelve al **Paso 3**, es decir el proceso se reinicia hasta que se verifique que $m(I_k) < \delta$ para algún valor de $k \in \mathbb{N}$.

Nota: Cada una de estas iteraciones reduce la longitud del rango de incertidumbre a la mitad. Por tanto, después de k iteraciones, se tiene que:

$$\boxed{m(I_k) = \left(\frac{1}{2}\right)^k \cdot m(I) \leq \delta} \tag{1}$$

donde: $m(I) = b - a$. Además, es de recordar que cada iteración requiere de dos evaluaciones de la función f.

Ejemplo 1. Búsqueda Dicotómica

Considere el siguiente problema de optimización:

$$\text{minimizar} \quad x^4 - \frac{1}{4}, \quad x \in \left[-\frac{1}{2}, 1\right].$$

Esta función, es claramente unimodal en toda la recta real, en particular lo es en $\left[-\frac{1}{2}, 1\right]$. Resulta sencillo observar que el óptimo se realiza en $x^* = 0$.

Se desea entonces aplicar el método de la búsqueda dicotómica para aproximar este valor de x^*. Para ello, se escoge $\varepsilon = 0.01$ (La cantidad ε que se ha elegido para construir los intervalos limita la precisión que se puede pedir, en este caso es bastante grande) y se inicia el proceso con $a = -\frac{1}{2}$ y $b = 1$, lo que resulta en que $m(I) = \frac{3}{2}$. Además, considere $\delta = 0.11$, entonces ¿cuántos iterados son necesarios como mínimo para tener la aproximación deseada? Esta respuesta es posible darla siguiendo la expresión dada en (1). A saber,

$$\left(\frac{1}{2}\right)^k \cdot \frac{3}{2} < 0.11 \;\rightarrow\; k \log\left(\frac{1}{2}\right) < \log\left(\frac{2 \cdot 0.11}{3}\right)$$
$$\rightarrow\; k < 3.77 \;\rightarrow\; \boxed{k > 4}$$

Por tanto, con 5 iterados como mínimo se tendrá la exactitud deseada. Se inicia el proceso:

1. $x_1 = \frac{1}{4} = 0.25$ y además, $x_a = \frac{6}{25} = 0.24$, $x_b = \frac{13}{50} = 0.26$, donde se tiene que: $f(x_a) \approx -0.2467$ y $f(x_b) \approx -0.2454$ $\rightarrow$ $f(x_a) < f(x_b)$.
   El nuevo Intervalo es: $I_1 = [-0.5, 0.26]$.

2. $x_2 = -0.12$ y además, $x_a = -0.13$, $x_b = -0.11$, donde se tiene que: $f(x_a) \approx -0.2497$ y $f(x_b) \approx -0.2498$ $\rightarrow$ $f(x_a) > f(x_b)$.
   El nuevo Intervalo es: $I_2 = [-0.13, 0.26]$.

3. $x_3 = 0.065$ y además, $x_a = 0.055$, $x_b = 0.075$, donde se tiene que: $f(x_a) \approx -0.24999$ y $f(x_b) \approx -0.24996$ $\rightarrow$ $f(x_a) < f(x_b)$.
   El nuevo Intervalo es: $I_3 = [-0.13, 0.075]$,

4. $x_4 = -0.0275$ y además, $x_a = -0.0375$, $x_b = -0.0175$, donde se tiene que: $f(x_a) \approx -0.249998$ y $f(x_b) \approx -0.249999$ ➔ $f(x_a) > f(x_b)$.

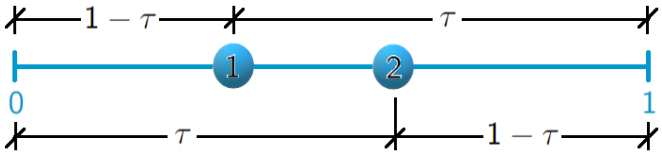
   El nuevo Intervalo es: $I_4 = [-0.0375, 0.075]$, donde $m(I_4) = 0.1125 > \delta$.

5. $\boxed{x_5 = 0.01875}$ y además, $x_a = 0.00875$, $x_b = 0.028753$, donde se tiene que: $f(x_a) \approx -0.2499999$ y $f(x_b) \approx -0.2499993$ ➔ $f(x_a) < f(x_b)$.

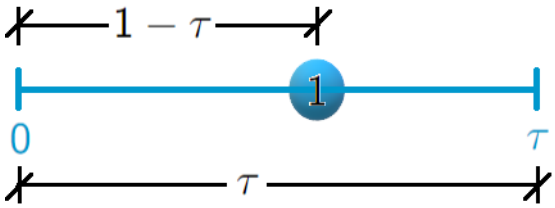
   El nuevo Intervalo es: $I_5 = [-0.0375, 0.028753]$, donde finalmente se cumple que: $m(I_5) = 0.06625 < \delta$. En consecuencia, $x^* \approx 0.01875$ ■

2.4 Búsqueda de la Sección Áurea

En la **sección Aurea** se ubican dos puntos interiores de manera tal que el intervalo eliminado en cada iteración sea de la misma proporción que el intervalo total y en este sentido, solo un nuevo punto debe ser calculado en cada iteración. Consideremos la localización simétrica de dos puntos como en la figura:



Se parte de un intervalo de longitud unidad (simplemente por conveniencia) y se localizan dos puntos, cada uno a una fracción τ de cada extremo. Con esta simetría, independientemente de qué valor de la función sea el más pequeño, la longitud del intervalo que permanece es siempre τ. Suponga que se elimina el subintervalo de la derecha:

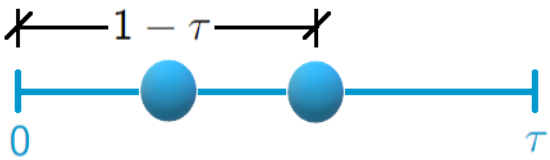


Se puede observar que el punto que queda, de los dos anteriores, está situado a una distancia $1 - \tau$ de uno de los extremos. En la estrategia que se plantea, el método del

número de oro el punto que permanece en el interior del nuevo intervalo está ubicado en la posición relativa en la que se encontraba el otro punto, que ahora limita la zona, la distancia $1-\tau$ debe corresponder a una fracción τ del intervalo (que es de longitud τ). Con esta elección de τ, el siguiente punto debe localizarse a una fracción τ de la longitud del intervalo desde el extremo de la parte derecha.

$$\frac{1-\tau}{\tau} = \frac{\tau}{1}$$

Por tanto, con la elección de τ que satisfaga $1-\tau = \tau^2$, el patrón de búsqueda permanece en el intervalo reducido de la siguiente figura.



La solución de esta ecuación cuadrática es justamente: $\boxed{\tau = \frac{-1 \pm \sqrt{5}}{2}}$, siendo la solución positiva de la misma $\tau \approx 0.61803$.

Forma de generar cada intervalo iterado

Dado un intervalo (a, b) se generan inicialmente dos nuevos puntos:

$$\boxed{y_1 = a + (1-\tau)(b-a)} \quad \text{y} \quad \boxed{z_1 = a + \tau(b-a)}$$

para algún $\tau \in \left(\frac{1}{2}, 1\right]$ convenientemente elegido.

Según los valores que tomen $f(y)$ y $f(z)$, el intervalo de incertidumbre en el siguiente paso será (a, z) o (y, b). Pues bien, la idea del método es elegir τ de forma que según el caso que corresponda el valor y_k coincida con el z_{k-1}, o bien z_k coincida con y_{k-1}.

Si $f(y_1) \geq f(z_1)$, entonces el nuevo intervalo de incertidumbre es $(a_2 = y_1, b_2 = b)$ y τ debe ser tal que:

$$\boxed{y_2 = a_2 + (1-\tau)(b-a_2) = z_1}$$

Puesto que $a_2 = y_1$, la relación anterior quedaría:

$$a + (1-\tau)(b-a) + (1-\tau)[b - (a + (1-\tau)(b-a))] = a + \tau(b-a)$$

$$2(1-\tau)(b-a) - (1-\tau)^2(b-a) = \tau(b-a)$$

Como $b \neq a$, se cumple que:

$$2(1-\tau) - (1-\tau)^2 = \tau \rightarrow 2 - 2\tau - 1 + 2\tau - \tau^2 = \tau$$

Luego,

$$\tau^2 + \tau - 1 = 0 \rightarrow \boxed{\tau = \frac{\sqrt{5}-1}{2}}$$

Nota: Observe que: $m(I_k) = \tau m(I_{k-1}) \rightarrow m(I_k) = \tau^k m(I_0)$.

Algoritmo de la sección áurea

- **Paso 1** **Valores de Entrada**

  Se fija la tolerancia en una valor $\delta > 0$ y se toma $\tau = \frac{\sqrt{5}-1}{2}$, donde $a_1 = a$ y $b_1 = b$.

- **Paso 2** **Calculos iniciales**

  (a) Calcular: $y_1 = a_1 + (1-\tau)(b_1 - a_1)$ y $z_1 = a + \tau(b_1 - a_1)$

  (b) Calcular: $f(y_1)$ y $f(z_1)$ y tomar $\boxed{k = 1}$

- **Paso 3** **Inicio del Proceso**

  Si $b_k - a_k < \delta \rightarrow$ **PARAR**. Entonces, $[a_k, b_k]$ es el intervalo buscado. En caso contrario, ir al **Paso 4**.

- **Paso 4** **Calculo de Imágenes**

  Calcular $f(y_k)$ y $f(z_k)$. Si $f(y_k) \geq f(z_k)$ ir al **Paso 5**. En caso contrario, ir al **Paso 6**

- **Paso 5** **Actualización de z_{k+1}**

  (a) Hacer $a_{k+1} = y_k$, $b_{k+1} = b_k$ y $y_{k+1} = z_k$, donde

$$z_{k+1} = a_{k+1} + \tau(b_{k+1} - a_{k+1})$$

(b) Calcular $f(z_{k+1})$. Hacer $k = k + 1$ (Actualizar eñl contador) e ir al **Paso 3**.

► **Paso 6** **Actualización de** y_{k+1}

(a) Hacer $a_{k+1} = a_k$, $b_{k+1} = z_k$ y $z_{k+1} = y_k$, donde

$$y_{k+1} = a_{k+1} + (1 - \tau)(b_{k+1} - a_{k+1})$$

(b) Calcular $f(y_{k+1})$. Hacer $k = k + 1$ (Actualizar eñl contador) e ir al **Paso 3**.

Utilice siguiente código en Matlab para realizar la búsqueda del valor que realiza el mínimo en el **Ejemplo 1**, considerando distintos valores para la tolerancia.

Método de la Sección Áurea

```
%% ALGORITMO PARA LA BUSQUEDA DE LA SECCION AUREA
%% PROBLEMA: Resuelva el problema de optimización
%%              min       x^4-0.25
%%            Sujeto a: -1/2<=x<=1
%%%
```

```
tau=0.618;
lambdak=tau*ak+(1-tau)*bk;
muk=(1-tau)*ak+tau*bk;
fl=(lambdak)^4-0.25;
fm=(muk)^4-0.25;
iter=0;
while abs(bk-ak)>=tol
    iter=iter+1;
    if fl>fm
        ak=lambdak;
        lambdak=muk;
        fl=fm;
        muk=(1-tau)*ak+tau*bk;
        fm=(muk)^4-0.25;
    elseif fl<fm
        bk=muk;
        muk=lambdak;
        fm=fl;
        lambdak=tau*ak+(1-tau)*bk;
        fl=(lambdak)^4-0.25;
    end
    D(iter,:)=[ak lambdak muk bk];
    plot(lambdak, fl, '*b', muk, fm, '*r')
    hold on
end
D
ak(end)
bk(end)
plot(x,y);
hold on
cputime
```

2.5 Búsqueda de Fibonacci

La idea de esta sección es tratar de mejorar sustancialmente la eficiencia de los métodos anteriores, reduciendo el número de evaluaciones de la función objetivo f. Recordamos que, en la búsqueda dicotómica por ejemplo, era necesario evaluar la función dos veces cada vez que desea obtener un nuevo intervalo.

En este sentido, el **método de Fibonacci** está considerado cómo el más eficiente entre los métodos de búsqueda. Difiere del método de la sección áurea, en el hecho de que el valor de τ no permanece constante en cada iteración y el número de iteraciones viene predeterminado en base al nivel de exactitud especificado por el usuario. En consecuencia, la idea de la **búsqueda de Fibonacci**, es conseguir un algoritmo de búsqueda que, esencialmente, cambie la manera de elegir los nuevos rangos de incertidumbre.

Para ello, se considera un intervalo de incertidumbre $I_k = [a_k, b_k]$ y se etiquetan dos puntos contenidos en el mismo, esto es $x_{a,k}, x_{b,k} \in I_k$. Una pregunta natural, puede darse en el sentido de determinar el cómo elegir el siguiente intervalo, de donde es posible observar como en el caso de la búsqueda dicotómica, los valores de la función en estos puntos ($f(x_{a,k})$ y $f(x_{b,k})$) se pueden utilizar para reducir el intervalo de incertidumbre:

$$I^a_{k+1} = [a_k, x_{b,k}] \text{ , si } f(x_{a,k}) \leq f(x_{b,k}),$$

y

$$I^b_{k+1} = [x_{a,k}, b_k] \text{ , si } f(x_{a,k}) > f(x_{b,k}).$$

Si I^b_{k+1} es el nuevo intervalo seleccionado, entonces contendrá a x^* y, además, el valor de f ya ha sido calculado en uno de los puntos interiores de I^b_{k+1}. Ahora, es posible evaluar f en otro punto interior de I^b_{k+1}, llamémoslo $x_{b,k+1}$. Ahora, se tienen dos puntos evaluados en f y que pertenecen al intervalo I^b_{k+1}. Iterando este proceso hasta conseguir un rango de incertidumbre suficientemente pequeño garantizará la obtención

del mínimo.

Tal y como se pretendía, la producción de un nuevo intervalo solamente requiere una evaluación de la función f. Notese que, por construcción, se tiene esta relación entre los intervalos:

$$I_k = I_{k+1}^a \cup I_{k+2}^b.$$

Repitiendo este proceso un número arbitrario de veces, se generará una sucesión de

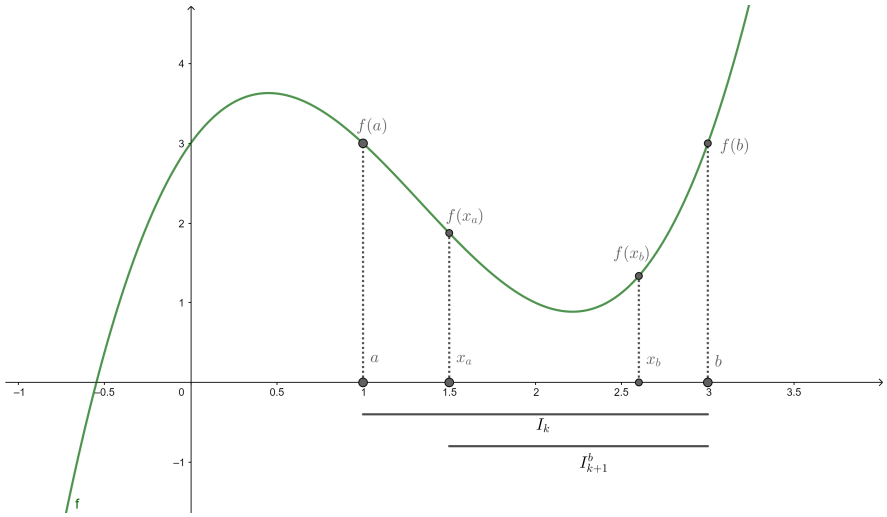


Figura 1: Esquema iterativo para el método de la búsqueda de Fibonacci.

intervalos $I_1, I_2, I_3, \ldots$ [2], tales que:

$$I_k = I_{k+1} \cup I_{k+2},$$

además, las longitudes de los intervalos estarán relacionadas de la siguiente manera:

$$m(I_k) = m(I_{k+1}) + m(I_{k+2}).$$

Dado n, se puede suponer que el intervalo I_{n+2} es el conjunto vacío, esto en función de que la longitud de los intervalos va reduciendo y a partir de algún n púiede ser despreciable dependiendo de la tolerancia pedida. Esto es, obviamente, una aproximación para hacer que la cadena de intervalos que se ha generado sea finita. Partiendo de esta premisa, se puede estudiar cómo evoluciona la longitud de los intervalos. Sea

[2] Notese que se han obviado los superíndices ya que, a priori, no se sabe de qué extremos se van a elegir para cada nuevo intervalo

$\omega_k = m(I_k)$ y en consecuencia, escribiendo de manera conveniente las últimas dos medidas, queda:

$$
\begin{array}{lcll}
\omega_{n+1} & = & \omega_n - \cancelto{0}{\omega_{n+2}} & = \boxed{1}\,\omega_n, \\
\omega_n & = & \omega_{n+1} + \cancelto{0}{\omega_{n+2}} & = \boxed{1}\,\omega_n, \\
\omega_{n-1} & = & \omega_n + \omega_{n+1} & = \boxed{2}\,\omega_n, \\
\omega_{n-2} & = & \omega_{n-1} + \omega_n & = \boxed{3}\,\omega_n, \\
\omega_{n-3} & = & \omega_{n-2} + \omega_{n-1} & = \boxed{5}\,\omega_n, \\
\omega_{n-4} & = & \omega_{n-3} + \omega_{n-2} & = \boxed{8}\,\omega_n, \\
 & \vdots & & \vdots \\
\omega_1 & = & \omega_2 + \omega_3 & = \boxed{F_n}\,\omega_n
\end{array}
$$

Nótese que los coeficientes que aparecen en cada iteración de ω_n en la parte derecha de cada ecuación, son justamente los números de Fibonacci $F_0 = 1$, $F_1 = 1$, $F_2 = 2$, $F_3 = 3$, $F_4 = 5, \ldots$. En general, estos números se pueden generar mediante la expresión recursiva:

$$F_k = F_{k-1} + F_{k-2}\,,\ k \geq 2$$

Se tiene entonces, que después de n iterados,

$$\boxed{\omega_n = \frac{\omega_1}{F_n}}$$

Para generar esta secuencia de intervalos, es necesario prefijar el número de iterados n. Suponga que se quiere obtener el óptimo con un error menor que una pequeña cantidad $\delta > 0$, entonces, se debe seleccionar n de manera que $\omega_1 < \delta F_n$. Tomar en cuenta estas consideraciones, es lo recomendable para esbozar un nuevo algoritmo de búsqueda.

Suponga que se tiene, como entrada del programa, un intervalo de incertidumbre inicial $[a_1, b_1]$, una función f unimodal y una tolerancia. La tolerancia fija el valor de n que hay que tomar. Como en el método de la búsqueda dicotómica hay que ir construyendo los intervalos sucesivos e ir eligiéndolos según el valor que tome la función f en sus puntos. En el estadio k del algoritmo, las siguientes cantidades serán conocidas: El intervalo de incertidumbre actual $[a_k, b_k]$; dos puntos interiores, $x_{a,k}$ y $x_{b,k}$; sus imágenes a través de la función f, $f_{a,k}$ y $f_{b,k}$ y la longitud del siguiente intervalo de

incertidumbre ω_{k+1}. Para que el algoritmo avance, se debe calcular el siguiente valor de cada una de esas cantidades. De la relación entre intervalos, se obtiene:

$$\omega_{k+2} = \frac{F_{n-k-1}}{F_{n-k}}\omega_{k+1}.$$

Por otro lado,

- Si $f_{a,k} > f_{b,k}$, entonces se escoge $a_{k+1} = x_{a,k}$, $b_{k+1} = b_k$. Se generan puntos interiores nuevos tomando $x_{a,k+1} = x_{b,k}$ y $x_{b,k+1} = x_{a,k+1} + \omega_{k+2}$. Además, $f_{a,k+1} = f_{b,k}$ (solamente es una asignación, no un cálculo) y $f_{b,k+1} = f(x_{b,k+1})$.
- Si $f_{a,k} < f_{b,k}$, los nuevos valores son $a_{k+1} = a_k$, $b_{k+1} = x_{b,k}$, $x_{a,k+1} = b_{k+1} - \omega_{k+2}$, $x_{b,k+1} = x_{a,k}$, $f_{b,k+1} = f_{a,k}$ y $f_{a,k+1} = f(x_{a,k+1})$.

El proceso se habrá completado para $k = n-2$, donde $\omega_{k+2} = \omega_n$ y la aproximación del óptimo vendrá dada por $x^* = x_{a,k+1} = x_{b,k+1}$ con un error no superior a $1/F_n$. Es bastante útil utilizar un último paso de búsqueda dicotómica para dividir el error por dos. Es posible asignar, pues, fijado $0 < \varepsilon < 1/F_n$

$$x^* = \begin{cases} x_{a,k+1} + \frac{1}{2F_n} & \text{, si } \; f(x_{a,k+1} + \varepsilon) < f(x_{a,k+1}), \\ x_{a,k+1} + \frac{\varepsilon}{2} & \text{, si } \; f(x_{a,k+1} + \varepsilon) = f(x_{a,k+1}), \\ x_{a,k+1} - \frac{1}{2F_n} & \text{, si } \; f(x_{a,k+1} + \varepsilon) > f(x_{a,k+1}). \end{cases}$$

Advertencia: Si el número necesario de iterados es muy grande, la diferencia entre $x_{a,k}$ y $x_{b,k}$ se puede hacer muy pequeña, haciendo problemático (debido a la propagación de errores) decidir cuál de los dos es más grande. Hay que programar el algoritmo teniendo en cuenta esto para evitar resultados erróneos. Una posibilidad es controlar la distancia entre $x_{a,k}$ y $x_{b,k}$ y terminar el programa cuando esta sea pequeña.

En resumen, se tiene que:

- Sea $I = [a, b]$ y se considera $a = a_1$ y $b = b_1$ ➜ $I = I_1$.
- Recuerde que: $\omega_k = m(I_k)$ ➜ $\boxed{\omega_1 = m(I_1) = b - a}$

- Se tiene entonces por construcción que:

$$\boxed{\omega_{k+2} = \frac{F_{n-k+1}}{F_{n-k}}\omega_{k+1}} \rightarrow \boxed{\omega_n = \frac{1}{F_n}\omega_1}$$

- Recuerde que la sucesión de Fibonacci está dada por:

$$\{1, 1, 2, 3, 5, 8, 13, 21, \ldots\} \rightarrow \boxed{F_k = F_{k-1} + F_{k-2}}$$

donde $F_0 = 1$ y $F_1 = 0$, para $k \geq 2$.

Algoritmo de la búsqueda de Fibonacci

Preliminares

- Sea $\delta > 0$ la tolerancia, tal que: $\boxed{\omega_n < \delta}$ (**Parada**)
- Sean a_1, b_1, tales que $I_1 = [a_1, b_1]$ $\rightarrow$ $\boxed{\omega_1 = b_1 - a_1}$

Algoritmo

❶ Se calcula F_k hasta que $\omega_1 < \delta F_n$, para algún n.

❷ Se calculan las cantidades $\omega_2, x_{a,1} = b_1 - \omega_2$, $x_{b,1} = a_1 + \omega_2$, y $f_{a,1} = f(x_{a,1})$ y $f_{b,1} = f(x_{b,1})$.

❸ Se actualizan los valores del nuevo intervalo.

❹ La condición de parada es $k = n - 2$ o $x_{a,k+1} > x_{b,k+1}$ y $x^* = x_{a,k+1}$.

Considere nuevamente el ejemplo anterior:

Ejemplo 2. Ejemplo para búsqueda de Fibonacci

Considere nuevamente el ejemplo anterior y calcule un iterado de la búsqueda de Fibonacci ($n = 3$).

$$\text{minimizar} \quad x^4 - \frac{1}{4}, \quad x \in \left[-\frac{1}{2}, 1\right].$$

Consideraciones Iniciales:

$$a_1 = -\frac{1}{2}, \; b_1 = 1 \quad \rightarrow \quad \boxed{\omega_1 = \frac{3}{2}}$$

Cálculo de ω_2. Veamos,

$$\omega_2 = \frac{F_2}{F_3}\omega_1 = \frac{2}{3} \cdot \frac{3}{2} = 1 \quad \rightarrow \quad \boxed{\omega_2 = 1}$$

Cálculo del nuevo intervalo:

$$x_{a,1} = b_1 - \omega_2 = 0 \quad \text{y} \quad x_{b,1} = a_1 + \omega_2 = \frac{1}{2}$$

Cálculo de las evaluaciones:

$$f_{a,1} = f(x_{a,1}) = -\frac{1}{4} = -0.25 \quad \text{y} \quad f_{b,1} = f(x_{b,1}) = -0.1875$$

Luego, el intervalo es justamente: $I_2^a = \left[0, \frac{1}{2}\right]$.

Nota: Se sabe de antes que el óptimo se alcanza en $x^* = 0$.

Accede al vídeo: La implementación de la búsqueda dicotómica

2.6 El método de la Interpolación Cuadrática

Como se ha visto, los métodos de búsqueda consisten en ir reduciendo adecuadamente el intervalo de incertidumbre en el que se encuentra el óptimo de la función. Los métodos de aproximación de la función se basan en, a partir de la evaluación de

la función en unos pocos puntos, aproximar ésta por un polinomio. En este sentido, se recupera la idea introducida al inicio de este tema en la que se argumentaba que una manera de aproximar el mínimo de una función es aproximar primero la propia función por otra más sencilla y calcular el mínimo de la aproximación.

La función más sencilla en la que se puede pensar si se pretende aproximar una función unimodal con un mínimo es un polinomio de segundo grado:

$$p(x) = a_0 + a_1 x + a_2 x^2,$$

donde a_0, a_1 y a_2 son constantes. Será requeriso que este polinomio interpole la función f en los extremos y un punto interior del rango de incertidumbre que, para evitar una notación demasiado pesada, será rebautizado como $I = [x_1, x_3]$, con $x_2 \in (x_1, x_3)$. Esto equivale a decir que se verifica el siguiente sistema de ecuaciones lineal:

$$p(x_i) = a_0 + a_1 x_i + a_2 x_i^2 = f_i, \quad i = 1, 2, 3,$$

con $f_i = f(x_i)$. Luego, se puede obtener un polinomio interpolador resolviendo un sistema lineal 3×3 y evaluando tres veces la función f.

La solución a este sistema viene dada por:

$$a_2 = \frac{[(x_3 - x_2)f(x_1) + (x_1 - x_3)f(x_2) + (x_2 - x_1)f(x_3)]}{\Delta}$$

$$a_1 = \frac{[(x_2^2 - x_3^2)f(x_1) + (x_3^2 - x_1^2)f(x_2) + (x_1^2 - x_2^2)f(x_3)]}{\Delta}$$

$$a_0 = \frac{[x_2 x_3(x_3 - x_2)f(x_1) + x_1 x_3(x_1 - x_3)f(x_2) + x_1 x_2(x_2 - x_1)f(x_3)]}{\Delta}$$

donde $\Delta = (x_1 - x_2)(x_2 - x_3)(x_3 - x_1)$. Como $p(x)$ tiene un óptimo en $\overline{x} = -\dfrac{a_1}{2a_2}$, se puede aproximar la ubicación del mínimo de $f(x)$ por:

$$x^* \approx -\frac{[(x_2^2 - x_3^2)f(x_1) + (x_3^2 - x_1^2)f(x_2) + (x_1^2 - x_2^2)f(x_3)]}{2[(x_3 - x_2)f(x_1) + (x_1 - x_3)f(x_2) + (x_2 - x_1)f(x_3)]}$$

Si la función f se puede aproximar bien, entonces $\overline{x}$ será una buena aproximación de

x^*. En haras de mejorar, siguiendo la misma lógica, se sustituirá uno de los extremos x_1 o x_3 (según los valores de $f(x_1)$ y $f(x_3)$) por $\overline{x}$ calculado mediante el vértice.

Después de un número suficientemente grande de iteraciones, el rango de incertidumbre estarán próximos al óptimo x^*. Consecuentemente, el polinomio interpolador p, será una aproximación del polinomio de Taylor de orden 2 de la función f alrededor de x^*.

Para tres puntos no alineados (x_1, y_1), (x_2, y_2) y (x_3, y_3) la función cuadrática que pasa por ellos tiene la expresión:

$$y = \frac{(x-x_2)(x-x_3)}{(x_1-x_2)(x_1-x_3)}y_1 + \frac{(x-x_1)(x-x_3)}{(x_2-x_1)(x_2-x_3)}y_2 + \frac{(x-x_1)(x-x_2)}{(x_3-x_1)(x_3-x_2)}y_3$$

Ejercicio 1. Calcula la función de interpolación cuadrática para los puntos : $(0, 2)$, $(1, 2)$ y $(2, 4)$. Una vez que tengas la expresión de la función obtén el valor de "y" esperado cuando $x = 1.5$. **Respuesta:** $p(x) = 2 - x + x^2$.

Algoritmo de Interpolación Cuadrática

Suponga que tiene una función real, f, unimodal en el intervalo $[a, b]$ y de una variable. Vamos a desarrollar el procedimiento suponiendo que el óptimo se trata de un mínimo.

Inicio Tomar $\delta > 0$ como cota del error de aproximación.

Algoritmo Este proceso debe repetirse:

❶ Seleccionar tres puntos $x_1, x_2, x_3 \in [a, b]$ tales que $f(x_2) < f(x_1)$ y $f(x_2) < f(x_3)$.

❷ Obtener el valor mínimo del polinomio de interpolación correspondiente a los puntos x_1, x_2, x_3. Dicho mínimo viene dado por la expresión $\overline{x} = -\frac{a_1}{2a_2}$.

❸ Si $|x_2 - x^*| < \delta$, **PARAR**. x^* se toma como aproximación al verdadero óptimo de la función. En caso contrario, ir al paso ④.

❹ Si $x_2 \leq x^*$, entonces tomar $a = x_1, b = x_2$ y volver al paso ①. En caso contrario, tomar $a = x_2, b = x_3$ y volver al paso ①

El siguiente es otro posible algoritmo basado en el mismo método:

Paso Inicial Seleccionar tres puntos $x_1, x_2, x_3 \in [a, b]$. Tomar $h > 0$ cota del error de aproximación. Hacer $k = 1$.

Algoritmo Este proceso debe repetirse:

❶ Obtener x^* el valor mínimo del polinomio cuadrático de interpolación correspondiente a los puntos x_1, x_2, x_3. Dicho mínimo viene dado por el eje de simetría. Sea $y_k = f(x^*)$. Si $k = 1$, ir al paso ③, en caso contrario ir al paso ②.

❷ Si $|y_k - y_{k-1}| < \delta$, **PARAR**. x^* se toma como aproximación al verdadero óptimo de la función. En caso contrario, ir al paso ③.

❸ Sea i tal que $f(x_i) = \text{máx}\{f(x_1), f(x_2), f(x_3)\}$. Hacer $x_i = x^*$ y volver al paso ①.

Ejemplo 3.

Consideramos el siguiente problema de optimización:

$$\text{minimizar} \quad 1 - \cos(x), \quad x \in \left[-\frac{\pi}{2}, \frac{\pi}{2}\right].$$

❶ Considera $x_1 = -\frac{\pi}{2} = -x_3$ y $x_2 = 0$.

❷ El sistema lineal a resolver es justamente: $\begin{bmatrix} 1 & -\frac{\pi}{2} & \frac{\pi^2}{4} \\ 1 & 0 & 0 \\ 1 & \frac{\pi}{2} & \frac{\pi^2}{4} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$

❸ De la segunda ecuación, claramente $\boxed{a_0 = 0}$. Lo que reduce el problema a:

$$\begin{bmatrix} -\frac{\pi}{2} & \frac{\pi^2}{4} \\ \frac{\pi}{2} & \frac{\pi^2}{4} \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Lo que resulta en: $\boxed{a_1 = 0}$ y $\boxed{a_2 = \frac{4}{\pi^2}}$, de donde: $\boxed{p(x) = \frac{4}{\pi^2}x^2}$ En este caso, por la simplicidad del ejemplo, el valor donde se alcanza el mínimo del primer polinomio interpolador coincide con el valor donde se alcanza el mínimo de la función objetivo ($x^* = 0$). Notese que $\frac{4}{\pi^2} \approx 0.40528$, mientras que el polinomio de Taylor de la función objetivo desarrollado en $x = 0$ y truncado a orden 2 es $x^2/2$, cuyo coeficiente cuadrático es 0.5. ■

2.7 El método de Interpolación Cúbica

Se puede generalizar el método de la sección anterior, utilizando un polinomio de orden 3. Esto es:

$$p(x) = a_0 + a_1 x + a_2 x^2 + a_3 x^3.$$

Una forma de determinar este polinomio, es justamente resolviendo el sistema de ecuaciones:

$$\begin{cases} a_0 + a_1 x_1 + a_2 x_1^2 + a_3 x_1^3 & = & f(x_1) \\ a_0 + a_1 x_2 + a_2 x_2^2 + a_3 x_2^3 & = & f(x_2) \\ a_0 + a_1 x_3 + a_2 x_3^2 + a_3 x_3^3 & = & f(x_3) \\ a_0 + a_1 x_4 + a_2 x_4^2 + a_3 x_4^3 & = & f(x_4) \end{cases} \tag{2}$$

En efecto, hay distintas fórmulas de interpolación cúbica. Como en el método de interpolación cuadrática, los coeficientes a_i se pueden determinar de manera que p tenga derivadas coincidentes con las de f. Como en un polinomio de grado 3 hay 4 coeficientes, entonces cuatro ecuaciones son necesarias para completar la caracterización del polinomio p, como se expresó antes.

Notese que la curva que describe un polinomio de tercer grado es, en general, distinta de uno de segundo. En efecto, si p es un polinomio de grado tres, éste puede exhibir un máximo y un mínimo. En cualquier caso, ambos extremos relativos vendrán dados por los ceros de la primera derivada. Igualando ésta a cero, se obtiene

$$p'(x) = a_1 + 2a_2 x + 3a_3 x^2 = 0.$$

Esta ecuación cuadrática se resuelve de manera trivial y tiene dos soluciones

$$\bar{x} = \frac{1}{3a_3}\left(-a_2 \pm \sqrt{a_2^2 - 3a_1 a_3}\right).$$

Como se busca un mínimo, la segunda derivada tiene que ser positiva en el valor crítico, esto es

$$p''(\bar{x}) = 2a_2 + 6a_3\bar{x} > 0.$$

Luego, de las dos posibles soluciones que se obtiene al resolver la ecuación cuadrática correspondiente a la primera derivada, se puede elegir la que cumpla la condición para la segunda. Esto determina el mínmo una vez se tengan las coeficientes del polinomio.

Como en el caso anterior, tenemos que encontrar cuatro ecuaciones que interrelacionen el polinomio p y la función f. Una de las múltiples posibilidades es resolver el sistema dado en (2) escogiendo $I = [x_1, x_3]$ como rango de incertidumbre inicial y $x_2 \in (x_1, x_3)$. Se puede obtener la ecuación restante igualando el valor de la derivada en uno de los puntos, x_1, por ejemplo:

$$p'(x_1) = a_1 + 2a_2x_1 + 3a_3x_1^2 = f'(x_1).$$

Utilizando la regularidad de la función

Hasta ahora, los métodos que se han usado solamente asumen que la función unimodal que se está minimizando es continua. No es descabellado suponer cierto grado de derivabilidad. En caso de que sea posible suponer derivabilidad de primer orden con continuidad, entonces la derivada en el mínimo será nula. Esto implica que, se puede reducir la búsqueda del mínimo a la búsqueda de un cero de la función derivada. Además, si existen derivadas de orden superior, se podrán utilizar métodos para buscar ceros con ordenes de convergencia mayores. Existe una cantidad infinita de métodos numéricos para calcular ceros de funciones unidimensionales. En este tema se repasarán tres métodos clásicos que, además, servirán para plantear ciertas cuestiones interesantes para los siguientes capítulos.

2.8 Método de la bisección

Suponga que $f \in \mathcal{C}^1(I)$ es una función unimodal. Como ya se ha argumentado, se puede reducir el problema de minimización a encontrar un cero de la derivada. El

método de la bisección se basa en el Teorema de Bolzano[3]. Dado un intervalo $[a_0, b_0]$, se evalúa la derivada de la función en sus extremos y suponga que la derivada toma signos distintos. Esto implica que la función es creciente en uno de los extremos y decreciente en el otro. Por tanto, el extremo relativo estará contenido en el intervalo.

Se define $c_0 = \dfrac{a_0 + b_0}{2}$, esto es, el punto medio del intervalo y se evalúa ahora la derivada en c_0. Si la derivada es nula, entonces c_0 es el mínimo.

Suponga que la derivada en c_0 es distinta de cero. Entonces es positiva o negativa. En cualquier caso, su signo será opuesto al de a_0 o al de b_0. El algoritmo se puede esquematizar de la siguiente manera:

1. Dados a_0 y b_0 se calcula $c_0 = \dfrac{a_0 + b_0}{2}$,
2. Se calcula el valor de $f'(c_0)$. Si $|f'(c_0)|$ es suficientemente pequeño (menor que una tolerancia dada), entonces el programa retorna c_0.
3. En caso contrario, se verifica el signo de $f(c_0)$ y se reemplaza a_0 o b_0 por c_0 de manera que el nuevo intervalo tome valores con signos opuestos en sus extremos.

La convergencia del método de la bisección es lineal. Esto es, el error absoluto se divide por dos a cada iterado. Por otro lado, como ventaja importante la convergencia es garantizada si las hipótesis iniciales se cumplen.

El Método de Newton

El **método de Newton** (o método de **Newton-Raphson**) es una de las más poderosas herramientas para encontrar ceros de una función. En general, este método exige que la función sea continuamente diferenciable para ser utilizado en pro de hallar un cero de una función.

[3] El teorema de Bolzano dice que si una función continua toma valores de distinto signo en los extremos de un intervalo, entonces, necesariamente hay un cero contenido en ese intervalo.

Considere el desarrollo de Taylor de primer orden en un entorno de $x_n \in \mathcal{F}$:

$$f(x) \approx f(x_n) + f'(x_n)(x - x_n)$$

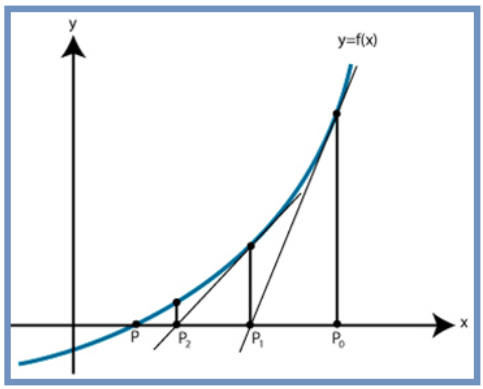
Al evaluar en x_{n+1}, queda:

$$f(x_{n+1}) = f(x_n) + f'(x_n)(x_{n+1} - x_n)$$

Si se asume que $x_{n+1} \to x^*$, entonces $f(x_{n+1}) \approx 0$.

En consecuencia, queda que:

$$\boxed{x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}}$$



En este caso, será utilizado para encontrar un cero de la primera derivada, luego,será necesario que la función a optimizar sea dos veces continuamente diferenciable, es decir que $f \in \mathcal{C}^2$.

Considerando el desarrollo en serie de Taylor de orden 2 de la función f alrededor de un punto x_0:

$$f(x_0 + h) = f(x_0) + f'(x_0)h + \frac{1}{2}f''(x_0)h^2 + \mathcal{O}(h^3).$$

La idea es simple, se trunca la expresión a orden 2 (esto es, eliminamos el término $\mathcal{O}(h^3)$ y se halla el mínimo del polinomio de orden 2 en h:

$$f(x_0) + f'(x_0)h + \frac{1}{2}f''(x_0)h^2.$$

En este caso, hay que calcular el mínimo de la parábola, que viene dado por

$$h = -\frac{f'(x_0)}{f''(x_0)}.$$

Salta a la vista que este razonamiento es problemático en caso de que la segunda

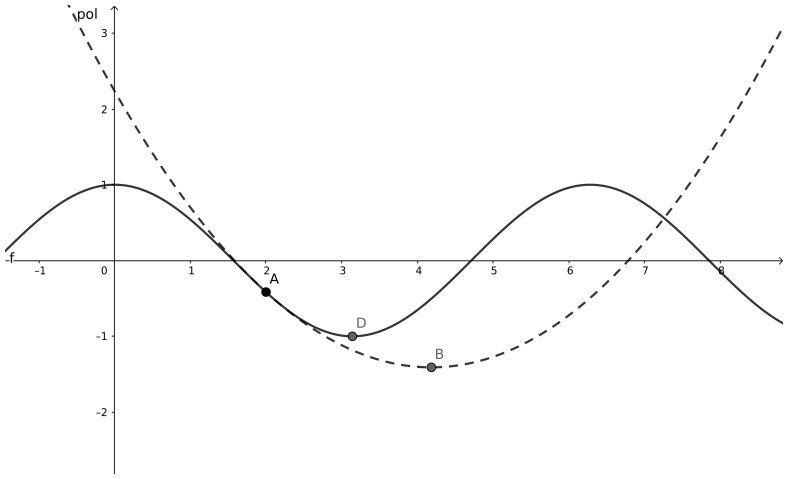


Figura 2: Parábola aproximando la función $\cos$ que es unimodal en $[0, 2\pi]$. El punto en que se desarrolla la serie de Taylor es $x = 2$ (etiquetado con la letra A). El extremo relativo de la función $\cos$ está marcado con la etiqueta D y el extremo relativo de la parábola está etiquetado con la letra B.

derivada sea nula en x_0, por eso se supondrá que el cero de la primera derivada sea simple. En caso contrario, uno debería adaptar el esquema utilizando una expansión en serie de Taylor de orden más alto.

El extremo relativo de la parábola es una aproximación del extremo relativo de la función original. Para mejorar esta aproximación se puede volver a tomar la parábola que aproxima este nuevo punto encontrado y repetir el proceso. Esto es, el esquema de Newton toma la forma:

$$\boxed{x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}}$$

El algoritmo iterativo construido con el esquema del método de Newton se desglosa en los siguientes puntos:

1. Se selecciona una condición inicial x_0.
2. Se evalúa el valor absoluto de la derivada $|f'(x_0)|$. Si éste es más pequeño que una tolerancia dada, entonces se retorna x_0.
3. Se actualiza $x_0 \leftarrow x_0 - \dfrac{f'(x_0)}{f''(x_0)}$ y se vuelve al punto **2**.

La convergencia del método de Newton es, en condiciones normales, cuadrática. Esto significa que es superior al método de la bisección. Por otro lado, la convergencia no está garantizada y depende de que el valor inicial x_0 se encuentre suficientemente cerca del óptimo. Es por eso que, para evitar que el algoritmo entre en un bucle infinito, hay que fijar un número de iteraciones máximas en el programa.

Ejemplo 4.

Suponga que se tiene el siguiente problema de optimización:

$$\text{minimizar} \quad x^4 + x^3 - 2x^2 - 4x + 1.$$

Esta función tiene un mínimo que se puede calcular a mano. Éste es:

$$x^* = -\frac{1}{4} + \frac{1}{12}\sqrt[3]{621 - 48\sqrt{87}} + \frac{\sqrt[3]{207 + 16\sqrt{87}}}{4\dot{3}^{2/3}} \approx 1.0666.$$

A continuación mostramos una implementación en Matlab del método de Newton. Las funciones definidas al inicio del código son la primera y la segunda derivada de la función en el **Ejemplo 4**.

Método de Newton

```
df = @(x) 4*x^3 + 3*x^2 - 4*x -4;
ddf= @(x) 12*x^2 + 6*x - 4;
x0=0.5;
maxiter=10;
tol=1E-12;
x(1)=x0;
fx(1)=df(x(1));
iter=1;
while(iter<maxiter)
      n1=df(x(iter));
      h=n1/ddf(x(iter));
```

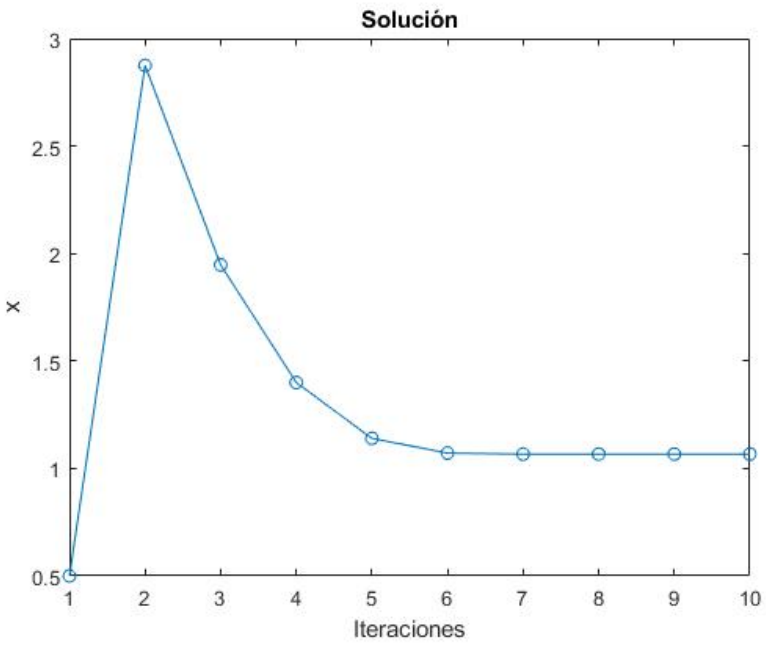


Figura 3: Representación de los iterados del método de Newton para el ejemplo (4). Se representa el iterado en el eje horizontal y la aproximación del óptimo para el eje vertical.

```
        x(iter+1)=x(iter)-h;
        sprintf('Iterado %d, aprox=%f, error=%e \n', ...
           iter, x(iter), n1)

        if( abs(n1)<tol || abs(h)<tol)
            iter=maxiter;
            break;
        end
        iter=iter+1;
end
```

2.9 El método de las Rectas Inexactas

En las técnicas de optimización multidimensional que se estudiarán en los próximos capítulos, es común que el esfuerzo computacional esté centrado en el cálculo del

gradiente de la función para así calcular la dirección del descenso al mínimo. Consecuentemente, la eficiencia de los métodos depende, en gran medida, de la capacidad que se tenga de calcular eficientemente el gradiente de la función con precisión. Si se requiere mucha precisión, el coste computacional del método crecerá. En algunos métodos, no es necesario calcular el gradiente con una gran precisión para obtener una convergencia rápida.

Considere el problema: $\min\limits_{x\in\mathbb{R}} f(x)$, donde f es una función diferenciable para todo $x \in \mathbb{R}$. Este problema, se puede resolver aplicando los métodos de descenso. Estos métodos, generan una secuencia de puntos que van reduciendo el valor de la función objetivo, o más formalmente se construye una sucesión de puntos $\{x_k\}_{k\geq 1}$, donde se cumple que:

$$f(x_1) > f(x_2) > f(x_3) > \cdots > f(x_n) > \cdots$$

lo cual, se termina para algún criterio de parada escogido.

La familia más importante de estos métodos, son los de **Dirección de Descenso**, los cuales constan de dos pasos fundamentales: Selección de una dirección de descenso y luego, calcular el desplazamiento a lo largo de esa dirección.

Un vector $\mathbf{d} \in \mathbb{R}^n$, es una dirección descenso de f a partir de $\mathbf{x} \in \mathbb{R}^n$, si la derivada direccional de f en la dirección de $\mathbf{x}$, es negativa, es decir:

$$\boxed{\nabla f(\mathbf{x})^T \cdot \mathbf{d} < 0}$$

En efecto, si $\mathbf{d} = -\nabla f(\mathbf{x})$, se tiene:

$$\nabla f(\mathbf{x})^T \cdot \mathbf{d} = -\nabla f(\mathbf{x})^T \cdot \nabla f(\mathbf{x}) = -\|\nabla f(\mathbf{x})\|^2 \leq \mathbf{0}$$

Luego, $\mathbf{d}$ es la dirección de máximo descenso, pero no es la única.

Caracterización: Si $\nabla f(\mathbf{x})^T \cdot \mathbf{d} < 0$, entonces:

$$\|\nabla f(\mathbf{x})^T\|\|\mathbf{d}\|\cos\theta < 0 \;\rightarrow\; \cos\theta < 0 \;\rightarrow\; \frac{\pi}{2} < \theta < \frac{3\pi}{2}$$

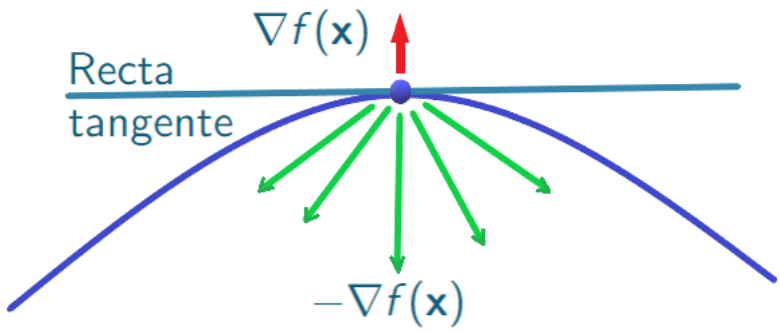


Figura 4: Representación de las direcciones de descenso, donde la másxima es justamente $-\nabla f(\mathbf{x})$.

Nota: Por ello, buscar un método para localizar la mejor dirección de descenso es importante.

Proposición 1

Sea $f : \mathbb{R}^n \longrightarrow \mathbb{R}$, $f \in \mathcal{C}^1$. Si existe $\mathbf{d} \in \mathbb{R}^n$, tal que $\nabla f(\mathbf{x})^T \cdot \mathbf{d} < 0$. Entonces, existe $\lambda > 0$ tal que: $f(\mathbf{x} + \lambda\mathbf{d}) < f(\mathbf{x})$. □

Observación: El problema radica en cómo escoger a λ.

Demostración

Sea $\varphi(\lambda) = f(\mathbf{x} + \lambda\mathbf{d}) \rightarrow \varphi(0) = f(\mathbf{x})$. Además,

$$\varphi'(\lambda) = \nabla f(\mathbf{x})^T \cdot \mathbf{d} \;\rightarrow\; \varphi'(0) = \nabla f(\mathbf{x})^T \cdot \mathbf{d} < 0$$

ya que, $\mathbf{d}$ es una dirección de descenso. En efecto,

$$\varphi'(0) = \lim_{\lambda \to 0} \frac{\varphi(\lambda) - \varphi(0)}{\lambda} < 0 \;\rightarrow\; \varphi(\lambda) < \varphi(0)$$

de donde, $f(\mathbf{x} + \lambda\mathbf{d}) < f(\mathbf{x})$. □

Algoritmo básico en la iteración k

Paso 1 Dado el punto x_k.

Paso 2 Calculamos la dirección de descenso $\mathbf{d}_k$.

Paso 3 Asignamos $x_{k+1} = x_k + \lambda \mathbf{d}_k$, para algún λ que haga "**aceptable**" a x_{k+1}

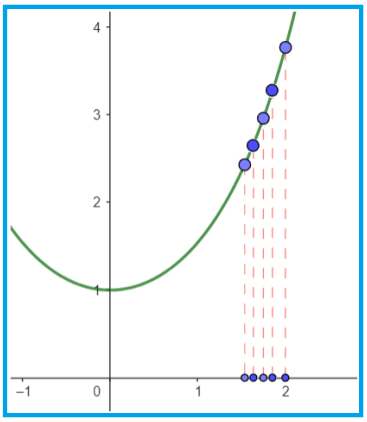
¿Importa la selección de la dirección de descenso?

Ejemplo 5. Dirección de descenso constante

Considere a la función $y = \cosh(x)$, donde $x_0 = 2$ y la dirección de descenso es constante $d_k = -1$ con $\lambda_k = 2^{-(k+1)}$. Trate de aproximar el valor donde se alcanza el mínimo ($x^* = 0$).

❶ Observe que en efecto d_k es una dirección de descenso

Iteración	x_k	d_k	λ_k	y_k
0	2.0000	-1	2^{-1}	3.7622
1	1.7500	-1	2^{-2}	2.9642
2	1.6250	-1	2^{-3}	2.6377
3	1.5625	-1	2^{-4}	2.4902
4	1.5312	-1	2^{-5}	2.4201
5	1.5156	-1	2^{-6}	2.3859



Nota: Parece que 1.5000 es una barrera. En realidad, el problema está en la elección de d y del paso, aun teniendo que:

❷ Sabemos que $y' = \text{senh}(x)$ ❸ $d_k y' < 0$, ya que $x > 0$. ■.

Ejemplo 6. Dirección de descenso alternada

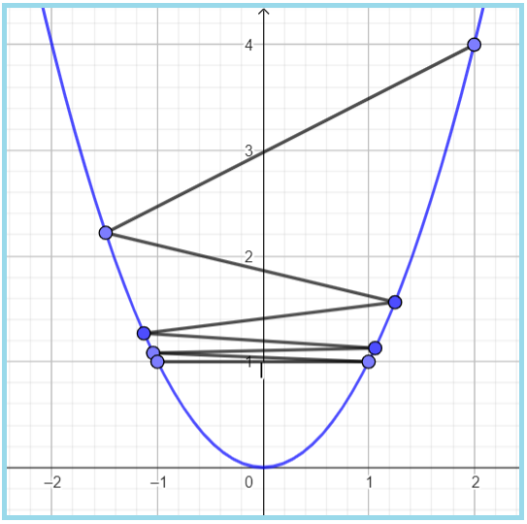
Considere ahora la función $f(x) = x^2$, con $x_0 = 2$ y con $d_k = -\text{sgn}(x_k)$ y el paso lo determina: $\lambda_k = 2 + 3 \cdot 2^{-k-1}$.

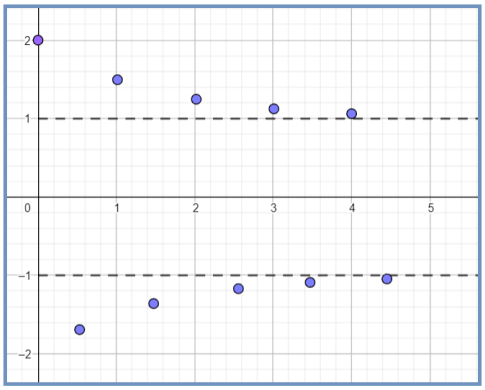
Haciendo un análisis, se puede constatar que a partir de cierto valor de $k \in \mathbb{N}$, la sucesión se comportará:

$$x_k = (-1)^k[1 + 2^{-k}] \rightarrow |x_{k+1}| < |x_k| \rightarrow x_{k+1}^2 < x_k^2$$

Luego, $f(x_{k+1}) < f(x_k)$ y esto verifica que la función decrece y tenemos un punto focal de un método para descender.

k	x_k	d_k	λ_k
0	2.00000	-1	3.500000
1	-1.50000	1	2.750000
2	1.25000	-1	2.375000
3	-1.12500	1	2.187500
4	1.06350	-1	2.093750
5	-1.03125	1	2.046875
$\vdots$	$\vdots$	$\vdots$	$\vdots$
48	-1.00000	-1	2.000000
49	1.00000	1	2.000000
50	-1.00000	-1	2.000000





⚠ ¿Qué sucede para $k \to \infty$?

$\lim\limits_{k\to\infty} x_k$ **No Existe**

¿Por qué el método no converge?

Ejemplo 7. Dirección de descenso y Paso

onsidere ahora la función $f(x) = x^2$, con $x_0 = 2$ y con $d_k = -1$ y el paso lo determinamos por: $\lambda_k = 2^{-k}$.

Observe que: $x_{k+1} = x_k + d_k\lambda_k$ ➔ $x_{k+1} = x_k - 2^{-k-1}$, de donde:

Iteración	x_k	d_k	λ_k	x_k
0	2	-1	2^{-1}	$2 = 1 + 2^0$
1	$\frac{3}{2}$	-1	2^{-2}	$\frac{3}{2} = 1 + 2^{-1}$
2	$\frac{5}{4}$	-1	2^{-3}	$\frac{5}{4} = 1 + 2^{-2}$
3	$\frac{9}{8}$	-1	2^{-4}	$\frac{9}{8} = 1 + 2^{-3}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$k+1$	x_k	-1	2^{-k-1}	$x_k = 1 + 2^{-k}$

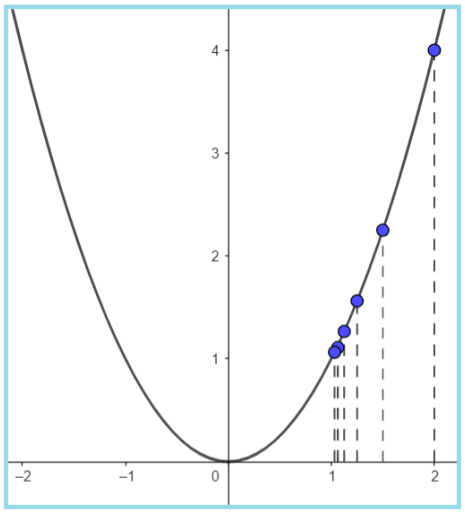
Observe que: $x_{k+1} < x_k$, lo que implica directamente que: $\boxed{f(x_{k+1}) < f(x_k)}$

Por tanto, disminuye siempre. Además,

$$\lim_{k\to\infty} x_k = 1 \neq 0 = x^*$$

¿El Problema? Los pasos son cada vez más pequeños y en efecto, aunque $\lambda_k > 0$ se tiene que:

$$\lim_{k\to\infty} \lambda_k = \lim_{k\to\infty} 2^{-k-1} = 0$$



En consecuencia, se deben evitar los pasos muy pequeños.

Observaciones:

- La dirección de descenso es un concepto local. Inspirado en Taylor.
- Esto asume pequeños pasos.
- Cuando hay muchos pasos el Teorema de Taylor deja de aplicarse.
- En cualquier dirección de ascenso, la función podría decrecer para muchos iterados.
- La solución es evitar muchos iterados.

Búsqueda Lineal

El proceso de búsqueda de λ, lo realizaremos en dos tandas: **Búsqueda lineal exacta** y la **Búsqueda lineal inexacta**. Veamos,

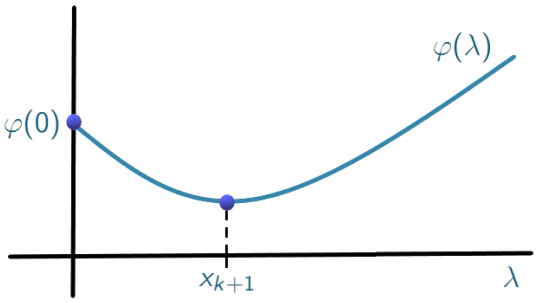
Búsqueda Lineal Exacta

Resulta necesario resolver el problema de minimización unidimensional:

$$\min_{\lambda \in \mathbb{R}} \varphi(\lambda)$$

O bien, hallar:

$$\lambda^* = \arg\min \varphi(\lambda)$$



Básicamente, hacemos $\varphi'(\lambda) = 0$ ➔ $\nabla f(\mathbf{x} + \lambda \mathbf{d})^T \cdot \mathbf{d} = 0$.

Nota: Para este proceso de búsqueda, se puede aplicar Fibonacci.

Ejemplo 8.

Volviendo al **Ejemplo 5** anterior, queda que:

- Se sabe que $f(x) = \cosh x$ ➔ $f'(x) = \text{senh} x$.
- Consideramos $d = -1$, con $x_0 = 2$.
- $f'(2 + \lambda(-1))(-1) = 0$ ➔ $-\text{senh}(2 - \lambda) = 0$ ➔ $\boxed{\lambda = 2}$

Entonces, con $\lambda = 2$ se hubiese resuelto el problema en un solo paso.

Búsqueda Lineal Inexacta (Algoritmo de Contracción de Paso)

Por lo general, se inicia el proceso con $\lambda = 1$ y si no se encuentra la dirección, este se va a ir encogiendo multiplicándolo por un factor $\alpha \in (0, 1)$.

Algoritmo de Contracción de Paso
Establece λ inicial: $\lambda = 1$ **y contractor** $\alpha = 0.5$
Mientras Condición sea Falsa realiza Actualiza $\lambda \leftarrow \alpha\lambda$
Imprime λ

Observación: No es posible que se haga un ciclo infinito.

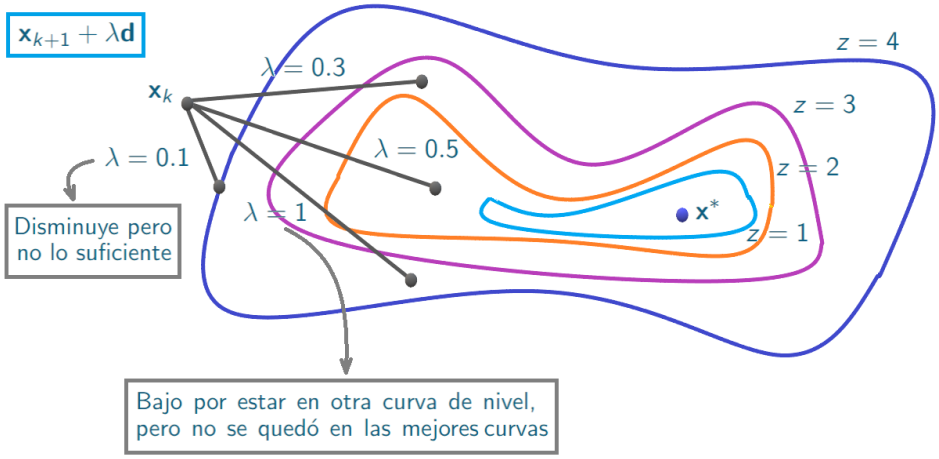


Figura 5: Efectos de la escogencia del paso λ

Condición de Armijo - Descenso suficiente

Sea $\alpha \in (0, 1)$. Se suele tomar inicialmente $\boxed{\alpha = 10^{-4}}$. Se sabe que el descenso siempre refiere a: $f(\mathbf{x} + \lambda \mathbf{d}) \leq f(\mathbf{x})$.

Para lo cual, **Larry Armijo** agregó:

$$\underset{\substack{\uparrow \\ \text{Buscado}}}{\boxed{f(\mathbf{x} + \lambda \mathbf{d})}} \leq \underset{\substack{\uparrow \\ \text{Descenso Siempre}}}{\boxed{f(\mathbf{x})}} + \underset{\substack{\uparrow \\ \text{Pequeño y negativo proporcional a } \nabla f(\mathbf{x})}}{\boxed{\alpha \lambda \nabla f(\mathbf{x})^T \cdot \mathbf{d}}}$$

Escrito en términos de φ, nos queda:

$$\underbrace{\varphi(\lambda)}_{\text{Curva}} \leq \underbrace{\varphi(0) + \alpha \lambda \varphi'(0)}_{\text{Recta de variable } \lambda}$$

La idea es que λ sea lo más grande posible y con esto se puedan dar pasos largos o mayores.

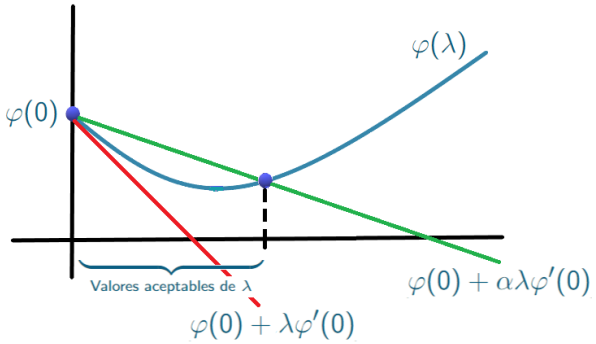


Figura 6: Valores aceptables de λ - Armijo

Condición de Goldstein - Condición de Curvatura

Condición propuesta por **Allen Goldstein**. Sea $\beta \in (\alpha, 1)$. Entonces,

$$\begin{array}{ccc} \boxed{\beta \nabla f(\mathbf{x})^T \cdot \mathbf{d}} & \leq & \boxed{\nabla f(\mathbf{x} + \lambda \mathbf{d})^T \cdot \mathbf{d}} \\ \downarrow & & \downarrow \\ \boxed{\beta \varphi'(0)} & \leq & \boxed{\varphi'(\lambda)} \end{array}$$

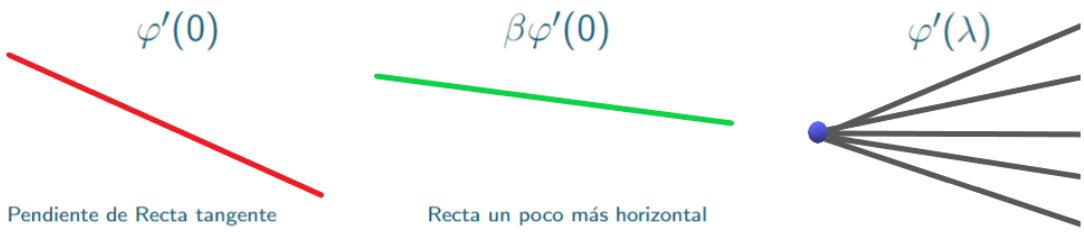


Figura 7: Escogencia del parámetros β - Goldstein

Permite que se utilicen valores de λ más grandes. En efecto, prohibe usar valores pequeños.

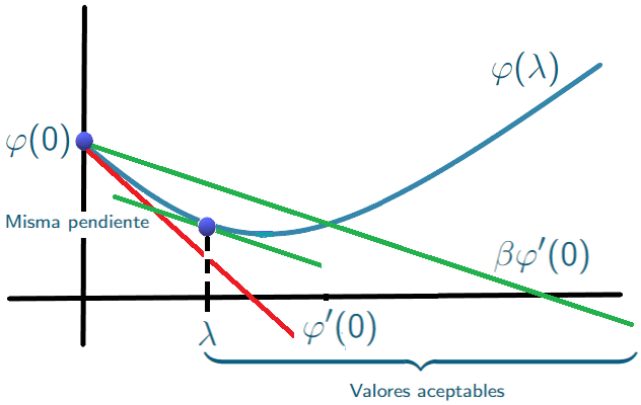


Figura 8: Valores aceptables para λ - Goldstein

Teorema 1: Wolfe - Phillip Wolfe - 1969

Sea $f : \mathbb{R}^n \longrightarrow \mathbb{R}$ continuamente diferenciable y sea $\mathbf{x}_k \in \mathbb{R}^n$ y $\mathbf{d}_k$ una dirección de descenso. Supongamos que para $\lambda > 0$ el conjunto $\{f(\mathbf{x} + \lambda \mathbf{d}_k)\}$ está acotado inferiormente. Si $0 < \alpha < \beta < 1$, entonces existen $\lambda_G, \lambda_A > 0$ con $\lambda_G < \lambda_A$, tales que: $\mathbf{x}_{k+1} = \mathbf{x}_k + \lambda \mathbf{d}_k$, satisface las condiciones de Armijo y de Goldstein, siempre que $\lambda \in (\lambda_G, \lambda_A)$. $\square$

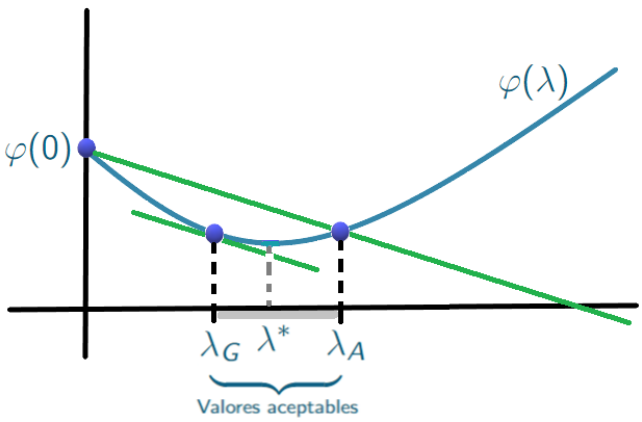


Figura 9: Valores aceptables para λ - Goldstein

Consideraciones iniciales más comunes:

$$\alpha \leftarrow 10^{-4}\,,\ \beta \leftarrow 10^{-1}\,,\ k \leftarrow 0\,,\ \varepsilon \leftarrow 10^{-6}\,,\ N \leftarrow 50$$

Establece $\mathbf{x}_0$ inicial

Mientras $\|\nabla f(\mathbf{x}_k)\| \geq \varepsilon$ y $k \leq N$ realiza

Calcular $\mathbf{d}_k$ (Búsqueda 1-dimensional)

$\lambda \leftarrow 1$

Mientras $f(\mathbf{x}_k + \lambda \mathbf{d}_k) > f(\mathbf{x}_k) + \alpha\lambda\nabla f(\mathbf{x}_k)^T \cdot \mathbf{d}_k$

y $\beta\nabla f(\mathbf{x}_k)^T \cdot \mathbf{d}_k > \nabla f(\mathbf{x}_k + \lambda \mathbf{d}_k)^T \cdot \mathbf{d}_k$

Actualiza $\lambda \leftarrow 0.5\,\lambda$

Fin

Actualiza $\mathbf{x}_k \leftarrow \mathbf{x}_k + \lambda \mathbf{d}_k$

Actualiza $k \leftarrow k + 1$

Fin

Imprime $\mathbf{x}_k$

Ejemplo 9. Condiciones de Wolfe

Determine el valor donde se alcanza el óptimo para la función: $f(x) = x^4 + x + 3$.

Paso 1 Iniciamos con el punto $x_0 = -1.5$ y $\lambda = 1$

Paso 2 Recordamos las condiciones de Wolfe (Armijo-Goldstein)

$$\begin{array}{lll} ❶ & \dashrightarrow & f(x_k+\lambda d_k) \leq f(x_k)+\alpha\lambda f'(x_k)d_k \\ ❷ & \dashrightarrow & \beta f'(x_k)d_k \leq f'(x_k+\lambda d_k)d_k \end{array}$$

Paso 3 Calculamos la derivada de f: $f'(x) = 4x^3 + 1$.

Paso 4 Elegimos las constantes: $\beta = 0.1$ y $\alpha = 0.9$, de forma arbitraria, donde $0 < \beta < \alpha < 1$.

Observemos que, para $\lambda = 1$ se verifican las dos condiciones del teorema de Wolfe:

$$\begin{aligned} ❶\ 2.5625 &= f(x_0+\lambda d_0) = f(-1.5+1\cdot 1) \\ &\leq f(x_0)+\alpha\lambda f'(x_0)d_0 = f(-1.5)+0.9\cdot 1\cdot f'(-1.5)\cdot 1 \\ &= 6.5625+0.9\cdot 12.5 = 17.8125 \end{aligned}$$

$$\begin{aligned} ❷\ -1.25 = 0.1\cdot 12.5\cdot 1 &= \beta f'(x_0)\cdot d_0 \\ &\leq f'(x_0+\lambda d_0)\cdot d_0 = f'(-1.5+1\cdot 1)\cdot 1 \\ &= f'(-0.5) = 0.5 \end{aligned}$$

Aceptamos el valor de $\lambda = 1$ y tomamos $x_1 = -1,5+1 = -0.5$.

Paso 5 Tomamos $x_1 = -0.5$, con $\lambda = 1$. $[d_1 = -\text{sgn}(f'(-0.5)) = -1]$.

Observemos que la primera condición de Wolfe (Armijo):

$$\begin{aligned} ❶\ 6.5625 &= f(x_1+\lambda d_1) = f(-0.5-1\cdot 1) \\ &\not\leq f(x_1)+\alpha\lambda f'(x_1)d_1 = f(-0.5)-0.9\cdot 1\cdot f'(-0.5)\cdot 1 \\ &= 2.5625-0.9\cdot 0.5 = 2.1125 \end{aligned}$$

no se verifica.

λ es demasiado grande: Tomemos $\lambda = 0.5$. En los dos siguientes iterados tampoco se verifica la primera condición de Wolfe, por lo que reduciremos dos

veces más hasta tomar el valor $\lambda = 0.125$.

$$f(x_1 + \lambda d_1) = 2.5276 \leq 2.56 = f(x_1) + \alpha\lambda d_1 f'(x_1),$$

y

$$d_1 f'(x_1 k + \lambda d_1) = -0.00234 \geq -0.05 = \beta d_1 f'(x_1)$$

Así pues, aceptamos $\lambda = 0.125$. Con lo que:

$$x_2 = -0.5 + (-1)0.125 = -0.625$$

Repitiendo el proceso obtenemos los valores $x_3 = -0.6328$ y $x_4 = -0.6289$.

Recuerde que estamos aproximando: $x^* = -\sqrt[3]{\frac{1}{4}} \approx -0.6299$.